

ПОЯСНЮВАЛЬНА ЗАПИСКА ДО ДИПЛОМНОГО ПРОЄКТУ

на тему: *«Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій»*

Київ – 2026

ЗМІСТ

ПЕРЕЛІК СКОРОЧЕНЬ.....	3
ВСТУП	4
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД ІСНУЮЧИХ РІШЕНЬ.....	6
1.1 Актуальність та доцільність розроблення.....	6
1.2 Огляд та аналіз існуючих рішень.....	7
1.2.1 Sprout Baby Tracker	7
1.2.2 Huckleberry	8
1.2.3 Glow Baby.....	9
1.3 Аналіз функціональності та вимог до системи	11
1.3.1 Порівняльна характеристика розглянутих рішень.....	11
1.3.2 Вимоги до реалізації застосунку	12
ВИСНОВОК ДО РОЗДІЛУ 1	14
РОЗДІЛ 2. ОГЛЯД ТА ВИБІР ТЕХНОЛОГІЙ ДЛЯ ПРОЕКТУВАННЯ СИСТЕМИ.....	15
2.1 Загальні принципи побудови кросплатформних мобільних застосунків.....	15
2.1.1 Клієнт-серверна архітектура мобільних застосунків.....	15
2.1.2 Основні компоненти мобільного застосунку.....	16
2.2 Огляд технологій для реалізації застосунку.....	17
2.2.1 Технології клієнтської частини.....	17
2.2.2 Хмарна платформа та серверна частина	18
2.2.3 Характеристика системи управління базами даних.....	19
2.2.4 Управління станом, локалізація та допоміжні бібліотеки ...	19
2.3 Механізми автентифікації та захисту даних.....	20
2.3.1 Row-Level Security та ізоляція даних користувачів	20
2.3.2 Безпека мобільного застосунку	21
2.4 Стандарти ВООЗ та педіатричний моніторинг	21
2.5 Інтеграція штучного інтелекту на основі RAG	22
2.5.1 Поняття векторного пошуку та RAG-архітектури.....	22
ВИСНОВОК ДО РОЗДІЛУ 2	24

					<i>ІАЛЦ.467200.003 ПЗ</i>			
<i>Зм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Підп.</i>	<i>Дата</i>				
<i>Розробив</i>	<i>Льоскін І. В.</i>				<i>Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Пояснювальна записка</i>	<i>Лит.</i>	<i>Аркуш</i>	<i>Аркушів</i>
<i>Перевірів</i>	<i>Кобилюк А. Г.</i>						1	28
<i>Н. контр.</i>	<i>Сімоненко В. П.</i>					<i>НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ 10-25</i>		
<i>Затвердив</i>	<i>Волокита А. М.</i>							

РОЗДІЛ 3. РЕАЛІЗАЦІЯ СИСТЕМИ	27
3.1 Архітектура системи.....	27
3.1.1 Загальна архітектурна схема застосунку	27
3.1.2 Архітектурні особливості клієнтської частини	27
3.1.3 Архітектура хмарної серверної частини	27
3.1.4 Архітектура бази даних.....	27
3.2 Реалізація серверної частини	27
3.2.1 Проєктування та реалізація бази даних.....	27
3.2.2 Реалізація Row-Level Security.....	27
3.2.3 Реалізація Edge Function та ІІІ-асистента.....	27
3.2.4 Реалізація системи автентифікації	27
3.2.5 Зберігання файлів та синхронізація в реальному часі	27
3.3 Реалізація клієнтської частини	27
3.3.1 Організація структури модулів	27
3.3.2 Навігаційна структура та управління станом	27
3.3.3 Реалізація ключових екранів та компонентів	27
3.3.4 Реалізація алгоритмів обробки медичних даних	27
ВИСНОВОК ДО РОЗДІЛУ 3	27
РОЗДІЛ 4. РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ СИСТЕМИ	27
4.1 Реалізація основних модулів системи.....	27
4.1.1 Модуль журналювання активностей.....	27
4.1.2 Модуль антропометричного моніторингу	27
4.1.3 ІІІ-асистент на основі RAG	27
4.2 Тестування системи	27
4.2.1 Юніт-тестування компонентів	27
4.2.2 Тестування бази даних	27
4.2.3 Наскрізне тестування	27
4.3 Розгортання та налаштування застосунку	27
ВИСНОВОК ДО РОЗДІЛУ 4	27
ВИСНОВКИ.....	27

ПЕРЕЛІК СКОРОЧЕНЬ

API	(Application Programming Interface) прикладний програмний інтерфейс
HTTPS	(HyperText Transfer Protocol Secure) захищений протокол передачі гіпертексту
JWT	(JSON Web Token) веб-токен JSON
RAG	(Retrieval-Augmented Generation) генерація з доповненим пошуком
REST	(Representational State Transfer) передача репрезентативного стану
RLS	(Row-Level Security) безпека на рівні рядків
SDK	(Software Development Kit) набір інструментів розробника
SQL	(Structured Query Language) мова структурованих запитів
UI	(User Interface) інтерфейс користувача
БД	база даних
ВООЗ	Всесвітня організація охорони здоров'я
СУБД	система управління базами даних
ШІ	штучний інтелект

ВСТУП

У сучасному суспільстві батьки та опікуни щодня стикаються з необхідністю систематичного відстеження медико-біологічних показників новонародженої дитини: режиму годування та сну, динаміки маси тіла й зросту, графіка вакцинацій і вікових досягнень. ВООЗ наголошує, що регулярний моніторинг фізичного розвитку протягом перших двох років життя є ключовим інструментом раннього виявлення відхилень і своєчасного медичного втручання. Попри широкий вибір мобільних застосунків для моніторингу немовляти, жоден із них не поєднує в собі повноцінного антропометричного моніторингу відповідно до стандартів ВООЗ, синхронізації між пристроями в реальному часі та інтелектуального асистента з перевіреною педіатричною базою знань. Це зумовлює актуальність розроблення комплексної кросплатформної системи, яка усуне зазначені недоліки.

Метою дипломного проєкту є розроблення кросплатформної клієнт-серверної системи агрегації та обробки медико-біологічних даних з використанням хмарних технологій — мобільного застосунку для відстеження розвитку немовляти на платформах iOS та Android. Об'єктом дослідження є процеси агрегації, зберігання та аналізу медико-біологічних даних про розвиток немовляти. Предметом дослідження є методи та інструменти розроблення кросплатформних мобільних застосунків із застосуванням хмарних технологій і ШІ.

У процесі виконання роботи буде проведено аналіз предметної області та огляд існуючих аналогів, сформульовано вимоги до системи. Також буде здійснено огляд та обґрунтування вибору технологічного стеку: React Native та Expo для клієнтської частини, Supabase з PostgreSQL для хмарного бекенду, Groq API для ШІ-асистента. Наступні етапи передбачають

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		4

проектування реляційної схеми бази даних із захистом на рівні рядків (RLS), реалізацію всіх модулів застосунку, інтеграцію ІІІ-асистента на основі архітектури RAG з векторним пошуком, а також тестування системи на рівнях юніт-тестів, тестів бази даних і наскрізних тестів.

Розроблений застосунок є актуальним практичним рішенням для широкого кола користувачів — від молодих батьків, які вперше стикаються з необхідністю ведення медичних записів дитини, до сімей, де обидва батьки потребують спільного доступу до актуальних даних у режимі реального часу. Застосунок розгорнуто на хмарній інфраструктурі та готовий до використання на обох платформах.

РОЗДІЛ 1

АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД ІСНУЮЧИХ РІШЕНЬ

1.1 Актуальність та доцільність розроблення

За даними ВООЗ, перші два роки життя дитини є найбільш інтенсивним періодом фізичного розвитку. Регулярний моніторинг маси тіла, зросту та окружності голови дозволяє педіатрам вчасно виявляти відхилення від вікових норм і призначати корекційні заходи на ранніх стадіях. Паралельно батьки щодня фіксують режим годування та сну, стан здоров'я, вакцинації та вікові досягнення — дані, що формують повноцінну картину розвитку дитини і є незамінними під час лікарських консультацій.

Попри важливість систематичного ведення таких записів, більшість батьків і сьогодні покладаються на паперові щоденники або стандартні нотатки смартфона. Такий підхід має суттєві обмеження: дані легко втрачаються, їх неможливо автоматично перетворити на динамічні графіки, а ведення записів одночасно двома батьками з різних пристроїв призводить до неузгодженостей і втрати актуальності даних. Крім того, паперові записи не дозволяють зіставити показники дитини зі стандартними перцентильними кривими, що унеможливорює самостійну оцінку динаміки розвитку між лікарськими візитами.

У відповідь на ці потреби ринок мобільних застосунків для батьків активно розвивається. Проте, як показує аналіз наявних рішень, переважна більшість із них охоплює лише окремі аспекти моніторингу і не забезпечує комплексного підходу. Жоден із популярних застосунків не поєднує повноцінного антропометричного моніторингу відповідно до стандартів

ВООЗ, синхронізації між пристроями в реальному часі та інтелектуального асистента з перевіреною педіатричною базою знань — що визначає практичну актуальність розроблення такої системи.

1.2 Огляд та аналіз існуючих рішень

На ринку мобільних застосунків для батьків існує декілька поширених рішень для відстеження розвитку немовляти. Їх функціональність варіюється від базового журналювання активностей до комплексного відстеження сну з аналітикою. Було розглянуто три найбільш популярних застосунки: Sprout Baby Tracker, Huckleberry та Glow Baby. Аналіз цих рішень дозволив виявити їхні ключові переваги та недоліки, а також визначити напрями для вдосконалення у розроблюваній системі.

1.2.1 Sprout Baby Tracker

Sprout Baby Tracker — один із найстаріших і найвідоміших застосунків для відстеження немовляти, доступний на платформах iOS та Android. Застосунок орієнтований на ведення повноцінного журналу активностей дитини з перших днів життя і дозволяє фіксувати годування (грудне молоко та суміш), епізоди сну, зміни підгузників, вимірювання маси тіла та зросту, а також переглядати хронологію подій у вигляді стрічки. Передбачена підтримка декількох профілів дітей в одному обліковому записі, що зручно для сімей із двома і більше дітьми.

До переваг Sprout Baby Tracker належать зрілий і зручний інтерфейс, широка функціональність журналювання та надійна робота без підключення до мережі. Графіки зростання дозволяють відстежувати динаміку антропометричних показників, однак порівняння з перцентильними кривими ВООЗ безпосередньо в застосунку відсутнє. Синхронізація між пристроями реалізована через хмарне резервне копіювання і не відбувається

в реальному часі, що ускладнює спільне ведення записів двома батьками. ІІІ-асистент або педіатрична база знань у застосунку відсутні.

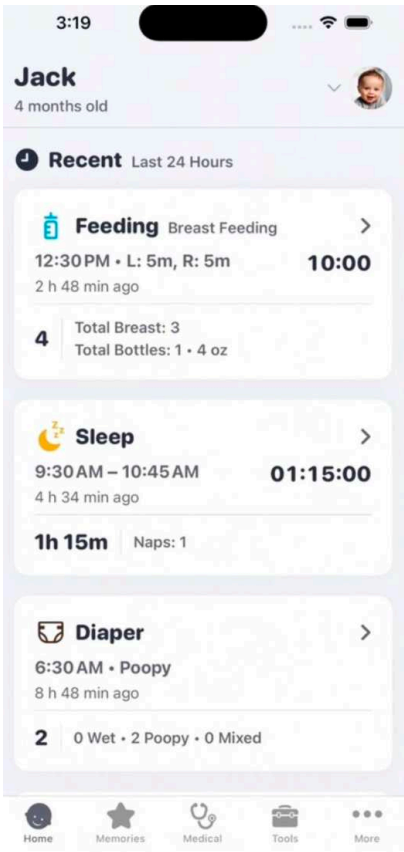


Рисунок 1.1 – Інтерфейс застосунку Sprout Baby Tracker

1.2.2 Huckleberry

Huckleberry — застосунок, орієнтований передусім на аналіз сну немовляти. Його ключова функція — алгоритм SweetSpot, який на основі накопичених даних про денний сон і нічний відпочинок пропонує оптимальний час для вкладання дитини. Застосунок збирає дані про тривалість і якість сну, фіксує годування та дозволяє вести загальні нотатки. Преміальна версія надає доступ до персоналізованих консультацій із сертифікованими фахівцями з дитячого сну.

Перевагою Huckleberry є глибока аналітика сну та наявність консультаційного сервісу, що виділяє його серед аналогів. Водночас застосунок має суттєві обмеження: повноцінний модуль антропометричного

моніторингу з перцентильними кривими ВООЗ відсутній, функціональність безкоштовної версії значно обмежена, а покрокового журналу активностей, який охоплював би всі аспекти догляду за немовлям, немає. ІІІ-асистент на основі бази педіатричних знань у застосунку не реалізований.

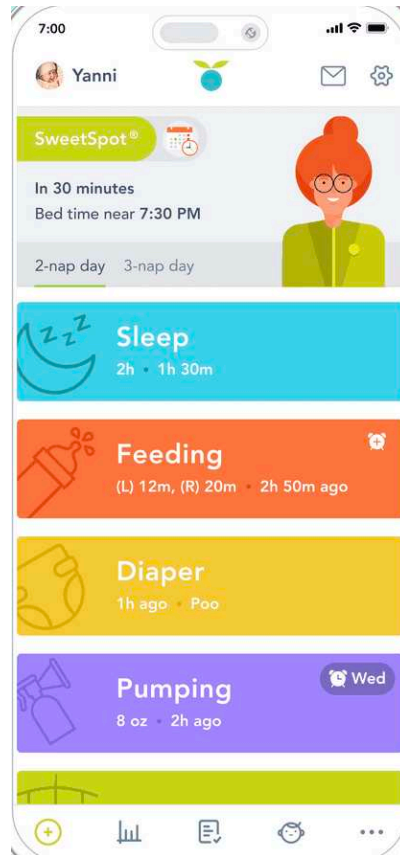


Рисунок 1.2 – Інтерфейс застосунку Huckleberry

1.2.3 Glow Baby

Glow Baby — комплексний трекер розвитку дитини від компанії Glow Inc., доступний на iOS та Android. Застосунок охоплює журнал активностей (годування, сон, підгузники, купання), відстеження вікових досягнень за категоріями (моторика, мова, соціальний розвиток), базові графіки зростання та широку базу статей про догляд за немовлятами. Передбачена функція спільного доступу, що дозволяє обом батькам переглядати та додавати записи.

Glow Baby вирізняється широтою контенту, системою вікових підказок і наявністю спільного доступу для батьків. Проте порівняння антропометричних показників із перцентильними кривими ВООЗ реалізоване спрощено і не відображає повний набір стандартів. Статті носять загальний інформаційний характер без прив'язки до конкретних даних дитини. Персоналізованого ШІ-асистента, здатного відповідати на педіатричні запитання на основі перевіреної бази знань, у застосунку немає. Синхронізація між пристроями в режимі реального часу також не забезпечена.

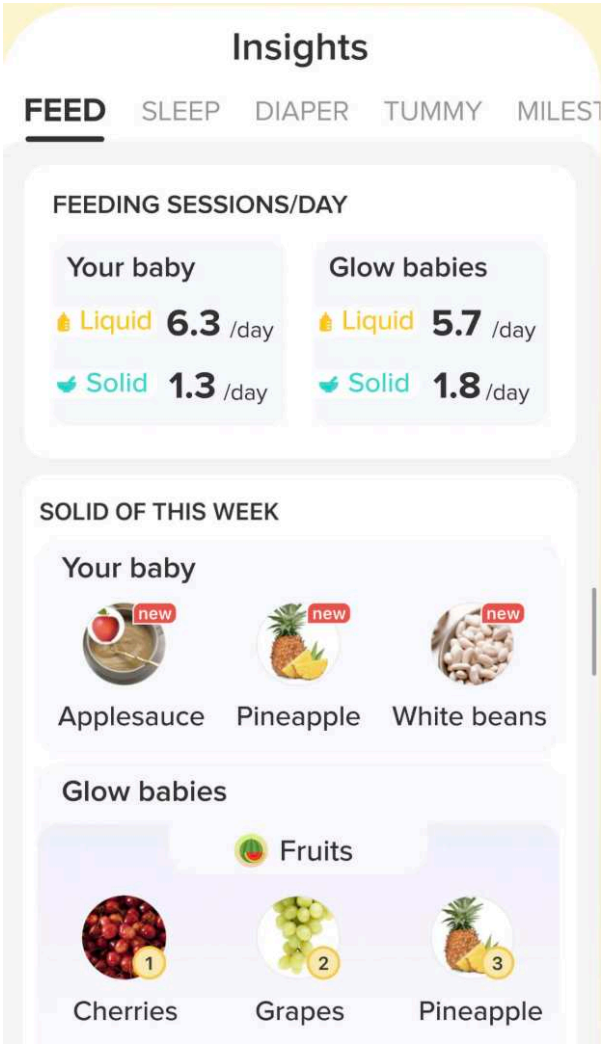


Рисунок 1.3 – Інтерфейс застосунку Glow Baby

1.3 Аналіз функціональності та вимог до системи

1.3.1 Порівняльна характеристика розглянутих рішень

На основі огляду трьох застосунків проведено порівняльний аналіз їхніх ключових переваг і недоліків. Результати наведено в таблиці 1.1.

Таблиця 1.1 – Порівняльна характеристика існуючих рішень

Застосунок	Переваги	Недоліки
Sprout Baby Tracker	1. Широкий журнал активностей 2. Підтримка декількох профілів дітей 3. Надійна робота без мережі	1. Відсутність кривих ВООЗ 2. Синхронізація не в реальному часі 3. Відсутність ШІ-асистента
Huckleberry	1. Глибока аналітика сну 2. Алгоритм SweetSpot 3. Чат з консультантом зі сну	1. Відсутність антропометричного модуля 2. Обмежений безкоштовний функціонал 3. Відсутність ШІ-асистента
Glow Baby	1. Широкий контент та вікові підказки 2. Спільний доступ для батьків 3. База статей про догляд	1. Спрощені графіки без кривих ВООЗ 2. Загальні статті без прив'язки до даних 3. Відсутність ШІ-асистента

Проведений порівняльний аналіз демонструє, що кожна з розглянутих платформ має свої сильні сторони, але водночас і певні обмеження. Sprout Baby Tracker вирізняється зрілим інтерфейсом та широким журналом активностей, але не має підтримки кривих ВООЗ та синхронізації в

реальному часі. Huckleberry пропонує унікальну аналітику сну, проте є вузькоспеціалізованим і не забезпечує комплексного моніторингу. Glow Baby охоплює найширший спектр функцій серед аналогів, однак поступається у точності антропометричних графіків і не має персоналізованого ШІ-асистента. Таким чином, жоден із розглянутих застосунків не поєднує повноцінного антропометричного моніторингу відповідно до стандартів ВООЗ, синхронізації в реальному часі та ШІ-асистента, що обґрунтовує доцільність розроблення нового рішення.

1.3.2 Вимоги до реалізації застосунку

З урахуванням аналізу існуючих рішень та потреб користувачів сформульовано функціональні та нефункціональні вимоги до розроблюваної системи. Вимоги формувалися з прагненням усунути виявлені недоліки аналогів і забезпечити комплексний інструмент для відстеження розвитку немовляти.

Функціональні вимоги:

- 1) Реєстрація та автентифікація користувача через email і пароль із захищеним зберіганням сесії.
- 2) Підтримка декількох профілів дітей та моделі опікунів: можливість запросити іншого користувача для спільного ведення записів.
- 3) Ведення журналу активностей: годування (грудне молоко, суміш, прикорм), сон, підгузники, купання, температура, ліки.
- 4) Додавання антропометричних вимірювань (маса тіла, зріст, окружність голови) та їх відображення на інтерактивних графіках із перцентильними кривими ВООЗ.
- 5) Управління графіком вакцинацій із відміткою виконаних щеплень.
- 6) Відстеження вікових досягнень дитини з відміткою дати виконання.

- 7) ШІ-асистент для відповідей на педіатричні питання на основі технології RAG з векторним пошуком у перевірній базі знань.
- 8) Синхронізація даних між пристроями в реальному часі через механізм підписки на зміни БД.

Нефункціональні вимоги:

- 1) Кросплатформна підтримка iOS 16.0+ та Android 10+ із нативним відображенням компонентів інтерфейсу.
- 2) Час відгуку API не більше 500 мс для основних операцій при стабільному мережевому з'єднанні.
- 3) Ізоляція даних між користувачами на рівні СУБД засобами RLS, незалежно від логіки клієнтського застосунку.
- 4) Масштабованість хмарної інфраструктури для обслуговування зростаючої кількості користувачів без зміни архітектури.
- 5) Захист застосунку від несанкціонованої модифікації та запуску на скомпрометованих пристроях.

Визначені функціональні та нефункціональні вимоги є основою для проєктування архітектури системи та вибору технологічного стеку в наступних розділах.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						13
Зм.	Лист	№ докум.	Підп.	Дата		

ВИСНОВОК ДО РОЗДІЛУ 1

У першому розділі проведено аналіз предметної області — системи агрегації та обробки медико-біологічних даних немовляти в мобільному застосунку. Обґрунтовано актуальність розроблення: регулярний педіатричний моніторинг є критично важливим у перші два роки життя дитини, проте більшість батьків не має зручного цифрового інструменту для його ведення з підтримкою реального часу та педіатричним ШІ-асистентом.

Здійснено огляд трьох популярних аналогів — Sprout Baby Tracker, Huckleberry та Glow Baby. Встановлено, що кожен із них орієнтований на певний аспект моніторингу: Sprout Baby Tracker охоплює широкий журнал активностей, Huckleberry спеціалізується на аналізі сну, а Glow Baby пропонує найбільший обсяг контенту серед аналогів. Водночас жоден із розглянутих застосунків не поєднує повноцінного антропометричного моніторингу з кривими ВООЗ, синхронізації між пристроями в реальному часі та ШІ-асистента з перевіреною педіатричною базою знань.

На основі порівняльного аналізу сформульовано вісім функціональних і п'ять нефункціональних вимог до розроблюваної системи. Функціональні вимоги охоплюють автентифікацію, модель опікунів, журнал активностей, антропометричний моніторинг із кривими ВООЗ, вакцинації, досягнення, ШІ-асистент та синхронізацію в реальному часі. Нефункціональні вимоги регламентують продуктивність, безпеку та масштабованість системи.

Визначені вимоги покладено в основу вибору технологічного стеку та проєктування архітектури системи, які розглядаються в наступних розділах.

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		14

РОЗДІЛ 2

ОГЛЯД ТА ВИБІР ТЕХНОЛОГІЙ ДЛЯ ПРОЕКТУВАННЯ СИСТЕМИ

2.1 Загальні принципи побудови кросплатформних мобільних застосунків

2.1.1 Клієнт-серверна архітектура мобільних застосунків

Більшість сучасних мобільних застосунків побудовано за клієнт-серверною архітектурою: клієнтська частина (мобільний застосунок) відповідає за відображення даних і взаємодію з користувачем, тоді як серверна частина забезпечує зберігання, обробку та ізоляцію даних. Такий поділ дозволяє масштабувати компоненти системи незалежно: бекенд може обслуговувати одночасно iOS- та Android-клієнтів без дублювання бізнес-логіки.

У контексті розроблення мобільних клієнтів виокремлюють три основні підходи, що відрізняються балансом між продуктивністю, витратами на розробку та доступом до платформних можливостей.

Нативна розробка передбачає окремі кодові бази для iOS (Swift/Objective-C) та Android (Kotlin/Java). Цей підхід забезпечує найвищу продуктивність, повний доступ до платформних API та найкраще відчуття платформи, однак вимагає підтримки двох незалежних кодових баз, що подвоює витрати на розробку та тестування.

Гібридний підхід базується на вебтехнологіях (HTML/CSS/JavaScript), загорнутих у нативну оболонку за допомогою таких фреймворків, як Ionic або Capacitor. Спільна кодова база суттєво знижує витрати, проте застосунок виконується у WebView, що призводить

до помітних затримок при рендерингу та нестандартної поведінки нативних елементів UI.

Кросплатформна нативна розробка поєднує переваги обох підходів: єдина кодова база транспілюється або компілюється у нативний код для кожної платформи. Серед представників цього підходу виокремлюють React Native від Meta та Flutter від Google. React Native відображає компоненти через нативний міст платформи, тоді як Flutter використовує власний рушій рендерингу Skia/Impeller. Обидва інструменти дозволяють досягти продуктивності, близької до нативної, при значно менших витратах на розробку.

Для розроблюваного застосунку обрано кросплатформну нативну розробку на базі React Native. Ключовими факторами вибору є зрілість екосистеми JavaScript/TypeScript, пряма інтеграція з Supabase SDK та Expo, а також можливість повторного використання вебкомпонентів. На відміну від Flutter, React Native не потребує вивчення мови Dart і органічно вписується в існуючу екосистему npm-пакетів.

2.1.2 Основні компоненти мобільного застосунку

Архітектура розроблюваного застосунку організована у чотири логічних шари з односпрямованою залежністю: вищий шар може звертатись до нижчого, але не навпаки, що запобігає циклічним залежностям і спрощує тестування.

Найближчим до користувача є шар представлення (*Presentation layer*), що містить компоненти UI, екрани та навігаційну структуру. У застосунку він реалізований на базі Expo Router із файловою маршрутизацією: кожен файл у директорії app/ автоматично відповідає окремому маршруту, а вкладеність директорій визначає ієрархію навігації. Компоненти цього шару отримують дані виключно через пропси або React-хуки і ніколи не звертаються до API

напряму.

Під ним розташований шар бізнес-логіки (*Business logic layer*), відповідальний за трансформацію даних, валідацію та управління станом застосунку. Він реалізований через кастомні React-хуки та глобальне сховище Zustand, у якому зберігаються поточний профіль дитини і стан автентифікаційної сесії. Цей шар виступає посередником між представленням і даними: він формує запити до API та передає готові до відображення об'єкти вгору по стеку.

Шар даних (*Data layer*) забезпечує взаємодію з хмарним API та управління локальним кешем. Він реалізований через клієнт Supabase у поєднанні з TanStack Query, що автоматично кешує відповіді сервера, виконує фонове оновлення при поверненні застосунку на передній план і повторює невдалі запити після відновлення мережевого з'єднання.

Основу стеку складає хмарний бекенд (*Backend layer*) — PostgreSQL з увімкненим RLS, автентифікаційний сервіс Supabase Auth, Realtime-підписки для синхронізації між пристроями та Edge Functions для додаткової серверної логіки. Бекенд не передбачає окремого серверного застосунку: бізнес-правила інкапсульовано безпосередньо у SQL-функції та тригери бази даних, що скорочує кількість рухомих частин системи.

2.2 Огляд технологій для реалізації застосунку

2.2.1 Технології клієнтської частини

React Native — фреймворк від компанії Meta для розроблення нативних мобільних застосунків із використанням JavaScript та TypeScript. На відміну від гібридних рішень, компоненти React Native транслюються безпосередньо у нативні елементи UI кожної платформи без проміжного WebView, що забезпечує продуктивність, порівнянну з нативними

застосунками. Єдина кодова база для iOS та Android у поєднанні з великою екосистемою npm-пакетів і підтримкою гарячого оновлення (Fast Refresh) суттєво прискорює цикл розробки.

Expo є офіційно рекомендованою надбудовою над React Native, що надає уніфікований SDK для доступу до нативних можливостей пристрою (камера, сповіщення, сенсори) без необхідності писати нативний код на Swift чи Kotlin. На відміну від bare React Native, Expo керує конфігурацією нативних проєктів автоматично через систему плагінів, що спрощує обслуговування і оновлення залежностей. Expo Router реалізує файлову маршрутизацію за аналогією з Next.js: структура директорій app/ безпосередньо визначає навігаційне дерево застосунку, усуваючи потребу в ручному описі маршрутів.

2.2.2 Хмарна платформа та серверна частина

Supabase — відкрита альтернатива Firebase на базі PostgreSQL. Платформа надає такі сервіси:

- реляційна база даних PostgreSQL з повноцінною підтримкою SQL;
- автентифікація (email/пароль, OAuth, magic link);
- REST та GraphQL API, автоматично згенеровані з схеми БД;
- Realtime — двостороння синхронізація змін у таблицях через WebSocket;
- Storage — зберігання файлів і зображень;
- Edge Functions — безсерверні функції на Deno для додаткової бізнес-логіки.

Supabase розгорнуто на хмарній інфраструктурі AWS і масштабується автоматично. Вибір Supabase замість Firebase обумовлений повноцінною підтримкою реляційних запитів, відкритим вихідним кодом та можливістю самостійного розгортання.

2.2.3 Характеристика системи управління базами даних

PostgreSQL — потужна об'єктно-реляційна СУБД з відкритим вихідним кодом. У контексті розроблюваної системи PostgreSQL відіграє центральну роль: зберігає всі медико-біологічні дані, забезпечує транзакційну цілісність і реалізує RLS.

Розширення pgvector додає підтримку векторних типів даних і індексів для пошуку найближчих сусідів у просторі ембедингів. Це дозволяє зберігати векторні представлення педіатричних статей безпосередньо в PostgreSQL і виконувати семантичний пошук у базі знань без зовнішнього векторного сховища.

2.2.4 Управління станом, локалізація та допоміжні бібліотеки

TanStack Query реалізує декларативне управління асинхронним станом: кешування, фонове оновлення, повторні спроби та синхронізацію запитів між компонентами. Бібліотека суттєво спрощує взаємодію клієнта з API та усуває потребу в ручному управлінні станами завантаження й помилок.

Zustand — мінімалістична бібліотека глобального стану для React та React Native. На відміну від Redux, вона не вимагає шаблонного коду і надає простий API для створення сховищ. У розроблюваному застосунку Zustand використовується для зберігання поточного профілю дитини та налаштувань сесії.

i18next — фреймворк інтернаціоналізації для JavaScript, що забезпечує підтримку декількох мов у застосунку. Застосунок підтримує українську та англійську локалізацію з автоматичним визначенням мови пристрою та можливістю ручного перемикавання.

react-native-gifted-charts — бібліотека графіків для React Native, що

використовується для відображення антропометричних даних дитини та стандартних кривих ВООЗ. Бібліотека надає гнучкий API для побудови лінійних і комбінованих графіків безпосередньо на нативному Canvas.

Groq API — платформа для швидкого LLM-inference, що надає доступ до великих мовних моделей (зокрема Llama 3.3 70B) через REST API з надзвичайно малою затримкою завдяки спеціалізованому апаратному прискорювачу LPU.

2.3 Механізми автентифікації та захисту даних

Безпека медико-біологічних даних дитини є критичною вимогою системи. Розроблювана архітектура реалізує захист на двох незалежних рівнях: на рівні транспорту всі запити передаються виключно через HTTPS, а на рівні СУБД доступ до рядків таблиць обмежується механізмом RLS.

Автентифікація здійснюється через Supabase Auth, що видає підписаний JWT-токен після успішного входу. Цей токен передається в заголовку кожного API-запиту та використовується СУБД для визначення поточного користувача.

2.3.1 Row-Level Security та ізоляція даних користувачів

RLS — механізм PostgreSQL, що дозволяє визначати умови видимості рядків таблиці на рівні СУБД. При увімкненому RLS кожен SQL-запит автоматично отримує додаткові умови фільтрації, визначені відповідними політиками безпеки.

У розроблюваній системі кожна таблиця (профілі дітей, активності, вимірювання, вакцинації) має набір RLS-політик, що дозволяють користувачу читати та змінювати лише власні записи. Ідентифікатор користувача витягується безпосередньо з JWT-токена функцією `auth.uid()`, що унеможливлює підробку ідентичності навіть за наявності прямого

доступу до API.

Таким чином, RLS надає додатковий рівень захисту, незалежний від логіки застосунку: навіть якщо клієнтський код містить помилку авторизації, СУБД не поверне чужих даних.

2.3.2 Безпека мобільного застосунку

Мобільний застосунок, що обробляє персональні медичні дані, є потенційною ціллю для реверс-інжинірингу та несанкціонованого доступу. Для захисту на рівні клієнтського середовища виконання інтегровано два інструменти.

freeRASP — бібліотека захисту мобільного застосунку в режимі виконання (*Runtime Application Self-Protection*). Вона виявляє загрозливе середовище: root-доступ або jailbreak на пристрої, підключені дебагери, запуск у емуляторі, а також ознаки переупакування або підробки застосунку. У разі виявлення загрози застосунок може заблокувати виконання критичних операцій або завершити сесію.

Sentry — платформа моніторингу помилок і продуктивності. Інтеграція Sentry дозволяє в реальному часі отримувати звіти про необроблені винятки, падіння застосунку та повільні мережеві запити. Це суттєво скорочує час виявлення та виправлення дефектів у виробничому середовищі.

2.4 Стандарти ВООЗ та педіатричний моніторинг

ВООЗ у 2006 році опублікувала Стандарти зростання дитини (*Child Growth Standards*) — набір перцентильних кривих, що описують нормальний фізичний розвиток дітей від народження до п'яти років. Стандарти побудовані на основі міжнародного мультицентрового дослідження за участю дітей із шести країн і вважаються глобальним еталоном.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						21
Зм.	Лист	№ докум.	Підп.	Дата		

Для кожного показника (маса тіла, зріст, окружність голови) та кожної статі визначено п'ять перцентильних ліній: P3, P15, P50, P85 та P97. Значення P50 відповідає медіані популяції, P3 — нижній межі норми, P97 — верхній. Відхилення вимірювання за межі P3 або P97 є підставою для консультації з педіатром.

У розроблюваному застосунку перцентильні таблиці ВООЗ вбудовано статично для діапазону 0–24 місяці окремо для хлопчиків і дівчаток. При додаванні нового вимірювання застосунок автоматично визначає перцентиль дитини і відображає її положення на графіку відносно кривих норми. Статичне вбудовування таблиць усуває залежність від мережевого підключення при перегляді антропометричних даних.

2.5 Інтеграція штучного інтелекту на основі RAG

Класичні LLM генерують відповіді виключно на основі знань, закладених під час навчання, і не мають доступу до актуальної або спеціалізованої інформації. Для вирішення цієї проблеми застосовується архітектура RAG.

2.5.1 Поняття векторного пошуку та RAG-архітектури

RAG — підхід, за якого перед генерацією відповіді система виконує семантичний пошук у базі знань і передає знайдені релевантні фрагменти безпосередньо в контекст запиту до мовної моделі. Це дозволяє моделі спиратися на актуальну та перевірену інформацію, а не лише на параметри навчання.

Семантичний пошук реалізується через векторне представлення тексту (*embeddings*): кожен фрагмент документа перетворюється на числовий вектор у багатовимірному просторі. Запит користувача також векторизується, після чого в базі знань шукаються фрагменти з найменшою косинусною

відстанню до вектора запиту.

У розроблюваній системі педіатричні статті та рекомендації зберігаються в PostgreSQL із розширенням pgvector. При надходженні запиту від користувача:

- 1) запит векторизується моделлю ембедингів;
- 2) у таблиці `knowledge_chunks` виконується пошук k найближчих фрагментів;
- 3) знайдені фрагменти разом із запитом передаються до Groq API;
- 4) мовна модель генерує відповідь, спираючись на наданий контекст.

Така архітектура забезпечує точні та верифіковані відповіді на педіатричні питання, мінімізуючи ризик галюцинацій (*hallucinations*) з боку мовної моделі.

ВИСНОВОК ДО РОЗДІЛУ 2

У другому розділі визначено технологічний стек та архітектурні принципи розроблюваної системи. Обґрунтовано вибір клієнт-серверної архітектури з чотирма логічними шарами: представлення, бізнес-логіки, даних і хмарного бекенду. Показано, що кросплатформна нативна розробка є оптимальним підходом для охоплення iOS та Android при єдиній кодовій базі.

Обрано React Native та Expo як основу клієнтської частини. React Native забезпечує нативне відображення компонентів без WebView, а Expo спрощує доступ до платформних можливостей і управління конфігурацією нативних проєктів через систему плагінів.

Хмарну платформу Supabase на базі PostgreSQL обрано завдяки підтримці реляційних запитів, механізму RLS для ізоляції даних між користувачами та вбудованому сервісу синхронізації в реальному часі. Розширення pgvector дозволяє зберігати векторні ембединги педіатричної бази знань безпосередньо в СУБД, без потреби у зовнішньому векторному сховищі.

Для управління станом обрано TanStack Query — для асинхронних запитів з автоматичним кешуванням, та Zustand — для глобального стану застосунку. Локалізацію забезпечує i18next з підтримкою української та англійської мов, а react-native-gifted-charts використовується для побудови антропометричних графіків із кривими BOOЗ. Захист виконуваного середовища реалізовано через freeRASP, моніторинг помилок — через Sentry.

Розглянуто стандарти BOOЗ щодо фізичного розвитку дітей і способів їх статичної інтеграції у застосунок у вигляді перцентильних таблиць для діапазону 0–24 місяці. Описано архітектуру ШІ-асистента на базі

					ІАЛЦ.467200.003 ПЗ	Аркуш
						24
Зм.	Лист	№ докум.	Підп.	Дата		